第 7 章 二叉树

1. 二叉树的概念

（可以用 ppt 演示）

1.1 二叉树的定义

二叉树是一种特殊的树型结构，它的特点是每个结点至多只有 2 棵子树（即二叉树中不存在度大于 2 的结点），并且二叉树的子树有左右之分，其次序不能任意颠倒。

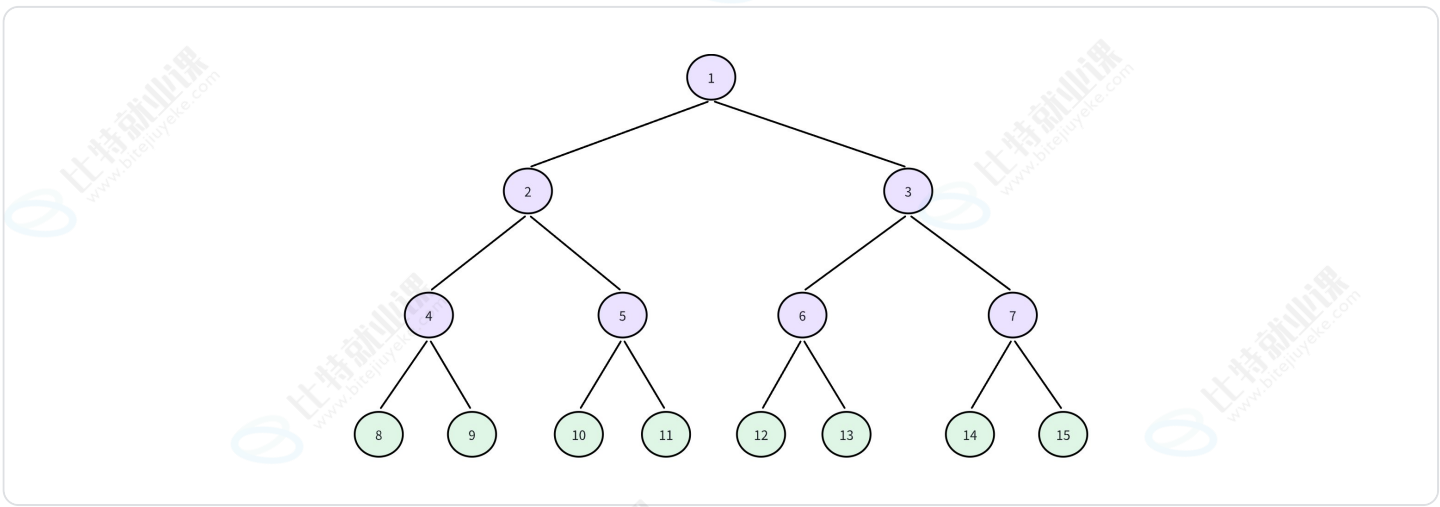
二叉的意思是这种树的每一个结点最多只有两个孩子结点。注意这里是最多有两个孩子， 也可能没有孩子或者是只有一个孩子。

> **注意：二叉树结点的两个孩子，一个被称为左孩子， 一个被称为右孩子。其顺序是固定的，就像人的左手和右手， 不能颠倒混淆。**

1.2 特殊的二叉树

满二叉树

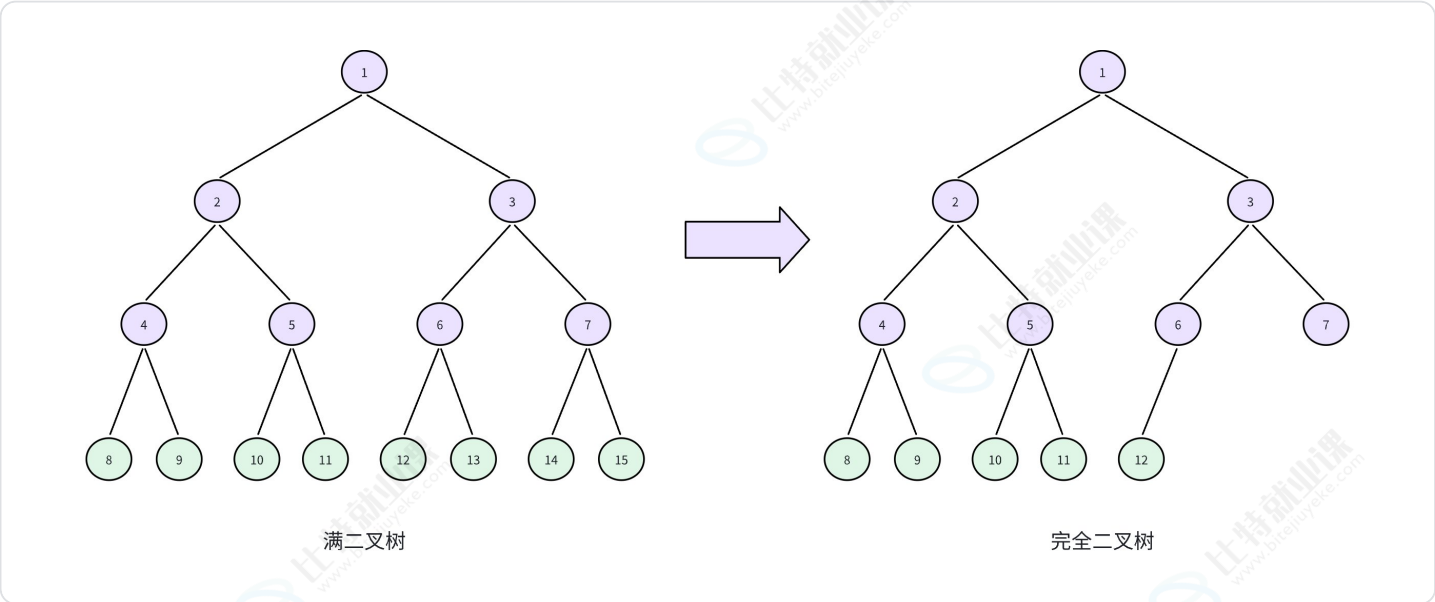
一棵二叉树的所有非叶子节点都存在左右孩子并且所有叶子节点都在同一层上，那么这棵树就称为满二叉树。



完全二叉树

对一棵树有 n 个结点的二叉树按层序编号，所有的结点的编号从 $1 \sim n$。如果这棵树所有结点和同样深度的满二叉树的编号为从 $1 \sim n$ 的结点位置相同，则这棵二叉树为完全二叉树。

说白了，就是在满二叉树的基础上，在最后一层的叶子结点上，从右往左依次删除若干个结点，剩下的就是一棵完全二叉树。



除了上述两种二叉树，还有堆、二叉排序树、平衡二叉树、红黑树等，会在后续课程中介绍。

2. 二叉树的存储

在《树》的章节中，已经学过树的存储，二叉树也是树，也是可以用**vector数组**或者**链式前向星**来存储。仅需在存储的过程中标记谁是**左孩子**，谁是**右孩子**即可。

- 比如用 vector 数组存储时，可以先尾插左孩子，再尾插右孩子；
- 用链式前向星存储时，可以先头插左孩子，再头插右孩子。只不过这样存储下来，遍历孩子的时候先遇到的是右孩子，这点需要注意。

但是，由于二叉树结构的特殊性，我们除了用上述两种方式来存储，还可以用符合二叉树结构特性的方式：分别是**顺序存储**和**链式存储**。

2.1 顺序存储

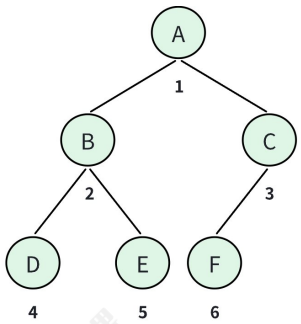
（可以用 ppt 演示）

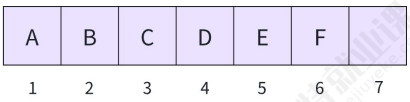
顺序结构存储就是使用数组来存储。

在完全二叉树以及满二叉树的性质那里，我们了解到：如果从根节点出发，按照层序遍历的顺序，由 1 开始编号，那么父子之间的编号是可以计算出来的。那么在存储完全二叉树的时候，就按照编号，依次放在数组对应下标的位置上，然后通过计算找到左右孩子和父亲：

> 结点下标为 i ：

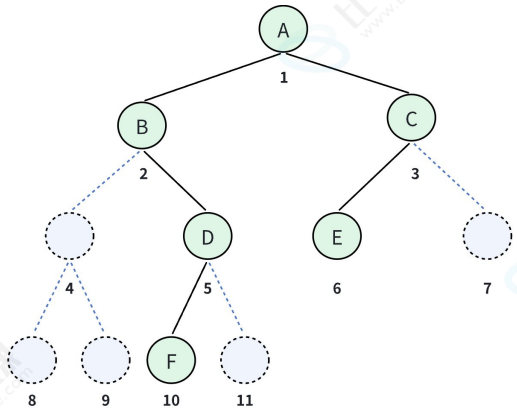
- 如果父存在，父下标为 $i/2$；
- 如果左孩子存在，左孩子下标为 $i \times 2$；
- 如果右孩子存在，右孩子下标为 $i \times 2 + 1$。

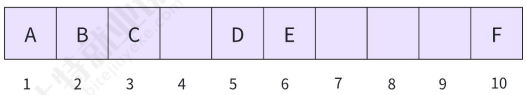




完全二叉树的顺序存储

如果不是完全二叉树，也是可以用顺序存储。但是首先要先把这棵二叉树补成完全二叉树，然后再去编号。不然就无法通过计算找到左右孩子和父亲的编号。

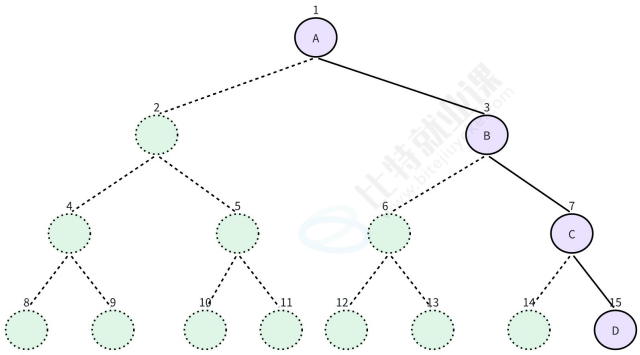




普通二叉树的顺序存储

可以看到我们的二叉树其实只有 6 个节点，但是顺序存储却要分配 10 个空间，其中有 4 个空间都被浪费掉了。

如下图我们考虑一种极端的情况，一棵树右斜树，它只有 4 个节点，但是需要分配 $2^4 - 1$ 个存储单元，这显然会对存储空间造成很大的浪费。所以，顺序存储结构一般只用于完全二叉树或满二叉树。



2.2 链式存储

链式存储类比链表的存储，都有静态实现和动态实现的方式。

我们这里依旧只讲**静态实现**，也就是用数组模拟。动态实现，就是 new 和 delete 的方式，会在就业课的数据结构课程中讲解。希望大家学有余力的情况下，可以把动态实现的方式也学习一下。因为找工作的笔试面试中遇到的二叉树，大多情况下会以动态的方式实现。

（以下内容可以用 ppt 演示）

竞赛中给定的树结构一般都是有编号的，参考上一章的树结构。因此我们可以创建两个数组 `l[N]`，`r[N]`，其中 `l[i]` 表示结点号为 i 的结点的左孩子编号，`r[i]` 表示结点号为 i 的结点的右孩子编号。这样就可以把二叉树存储起来。

案例：

题目描述：

> 有一个 $n(n \leq 10^6)$ 个结点的二叉树。给出每个结点的两个子结点编号（均不超过 n），建立一棵二叉树（根节点的编号为 1），如果是叶子结点，则输入 0 0。

输入描述：

> 第一行一个整数 n，表示结点数。
>
> 之后 n 行，第 i 行两个整数 l 、r，分别表示结点 i 的左右子结点编号。若 $l=0$ 则表示无左子结点，$r=0$ 同理。

代码实现：

```cpp
#include <iostream>

using namespace std;

```

```cpp
const int N = 1e6 + 10;

int n;
int l[N], r[N];

int main()
{
    cin >> n;
    for(int i = 1; i <= n; i++)
    {
        // 存下 i 号结点的左右孩子
        cin >> l[i] >> r[i];
    }

    return 0;
}
```

3. 二叉树的遍历

3.1 深度优先遍历

（可以用 ppt 演示）

不同于常规树的深度优先遍历，二叉树因其独特的性质可以划分成三种深度优先遍历：先序遍历，中序遍历，和后序遍历。其中，三种遍历方式的不同在于处理根节点的时机。

对于一棵二叉树而言，整体可以划分成三部分：根节点 + 左子树 + 右子树：

- 先序遍历的顺序为：根 + 左 + 右；
- 中序遍历的顺序为：左 + 根 + 右；
- 后序遍历的顺序为：左 + 右 + 根。

案例：

题目描述：

> 有一个 $n(n \leq 10^6)$ 个结点的二叉树。给出每个结点的两个子结点编号（均不超过 n），建立一棵二叉树（根节点的编号为 1），如果是叶子结点，则输入 0 0。

输入描述：

> 第一行一个整数 n，表示结点数。
>
> 之后 n 行，第 i 行两个整数 l 、r，分别表示结点 i 的左右子结点编号。若 $l = 0$ 则表示无左子结点，$r = 0$ 同理。

测试用例：

```
测试一：
4
0 2
3 4
0 0
0 0

测试二：
2
2 0
0 0

测试三：
3
2 3
0 0
0 0

测试四：
7
2 3
0 4
5 6
0 0
0 0
7 0
0 0
```

代码实现：

```cpp
#include <iostream>

using namespace std;

const int N = 1e6 + 10;

int n;
int l[N], r[N];

void dfs1(int p)
{
```

```cpp
    if(p == 0) return;

    // 先处理根节点
    cout << p << " ";
    // 左子树
    dfs1(l[p]);
    // 右子树
    dfs1(r[p]);
}

void dfs2(int p)
{
    if(p == 0) return;

    dfs2(l[p]);
    cout << p << " ";
    dfs2(r[p]);
}

void dfs3(int p)
{
    if(p == 0) return;

    dfs3(l[p]);
    dfs3(r[p]);
    cout << p << " ";
}

int main()
{
    cin >> n;
    for(int i = 1; i <= n; i++)
    {
        cin >> l[i] >> r[i];
    }

    dfs1(1); // 先序遍历
    cout << endl;

    dfs2(1); // 中序遍历
    cout << endl;

    dfs3(1); // 后序遍历
    cout << endl;

    return 0;
}
```

3.2 宽度优先遍历

这个就和常规的树的遍历方式一样，直接用队列帮助层序遍历即可。

代码实现：

```cpp
#include <iostream>
#include <queue>

using namespace std;

const int N = 300;

int n;
int l[N], r[N];

void bfs()
{
    queue<int> q;
    q.push(1);

    while(q.size())
    {
        auto p = q.front(); q.pop();
        cout << p << " ";

        // 左右孩子入队
        if(l[p]) q.push(l[p]);
        if(r[p]) q.push(r[p]);
    }
    cout << endl;
}

int main()
{
    cin >> n;
    for(int i = 1; i <= n; i++)
    {
        cin >> l[i] >> r[i];
    }

    bfs();
```

```
37
38      return 0;
39  }
```

4. 二叉树算法题

4.1 新二叉树

> **题目来源：** 洛谷
>
> **题目链接：** P1305 新二叉树
>
> **难度系数：** ★

【题目描述】

> 输入一串二叉树，输出其前序遍历。

【输入描述】

> 第一行为二叉树的节点数 n 。 $(1 \leq n \leq 26)$
>
> 后面 n 行，每一个字母为节点，后两个字母分别为其左右儿子。特别地，数据保证第一行读入的节点必为根节点。
>
> 空节点用 `*` 表示

【输出描述】

> 二叉树的前序遍历。

【示例一】

> 输入：
>
> ```
> 6
> abc
> bdi
> cj*
> d**
> i**
> j**
> ```
>
> 输出：
>
> ```
> abdicj
> ```

【解法】

1. 建树：和常规的链式存储方式一致。

因为结点是字符，所以可以直接用 `ASCII` 码值当做下标来使用。比如 `'a'` 直接映射成 97，$l[97]$ 里面就存着 `'a'` 的左儿子，$r[97]$ 里面就存着 `'a'` 的右儿子，以此类推，建立二叉树。

2. 先序遍历：根左右。

【参考代码】

```cpp
#include <iostream>

using namespace std;

const int N = 300;

int n;
char root;
char l[N], r[N]; // 存二叉树

void dfs(char root)
{
    if(root == '*') return;

    cout << root;
    dfs(l[root]);
    dfs(r[root]);
}

int main()
{
    cin >> n;
    // 处理根节点
    cin >> root;
    cin >> l[root] >> r[root];

    for(int i = 2; i <= n; i++)
    {
        char t; cin >> t;
        cin >> l[t] >> r[t];
    }

    // 前序遍历
    dfs(root);

```

```
36        return 0;
37    }
```

4.2 二叉树的遍历

题目来源： 洛谷

题目链接： B3642 二叉树的遍历

难度系数： ★

【题目描述】

有一个 $n(n \leq 10^6)$ 个结点的二叉树。给出每个结点的两个子结点编号（均不超过 n），建立一棵二叉树（根节点的编号为 1），如果是叶子结点，则输入 `0 0`。

建好树这棵二叉树之后，依次求出它的前序、中序、后序列遍历。

【输入描述】

第一行一个整数 n，表示结点数。

之后 n 行，第 i 行两个整数 l 、r，分别表示结点 i 的左右子结点编号。若 $l = 0$ 则表示无左子结点，$r = 0$ 同理。

【输出描述】

输出三行，每行 n 个数字，用空格隔开。

第一行是这个二叉树的前序遍历。

第二行是这个二叉树的中序遍历。

第三行是这个二叉树的后序遍历。

【示例一】

输入：

```
7
2 7
4 0
0 0
0 3
0 0
0 5
6 0
```

> 输出：
>
> 1 2 4 3 7 6 5
>
> 4 3 2 1 6 5 7
>
> 3 4 2 5 6 7 1

【解法】

建图 + dfs。

【参考代码】

```cpp
#include <iostream>

using namespace std;

const int N = 1e6 + 10;

int n;
int l[N], r[N]; // 存二叉树

void dfs1(int root)
{
    if(!root) return;

    cout << root << " ";
    dfs1(l[root]);
    dfs1(r[root]);
}

void dfs2(int root)
{
    if(!root) return;

    dfs2(l[root]);
    cout << root << " ";
    dfs2(r[root]);
}

void dfs3(int root)
{
    if(!root) return;

    dfs3(l[root]);
    dfs3(r[root]);
```

```
34        cout << root << " ";
35    }
36
37    int main()
38    {
39        cin >> n;
40        // 建树
41        for(int i = 1; i <= n; i++) cin >> l[i] >> r[i];
42
43        // 前序遍历
44        dfs1(1);
45        cout << endl;
46
47        // 中序遍历
48        dfs2(1);
49        cout << endl;
50
51        // 后序遍历
52        dfs3(1);
53
54        return 0;
55    }
```

4.3 二叉树深度

> **题目来源：** 洛谷
>
> **题目链接：** P4913 【深基16.例3】二叉树深度
>
> **难度系数：** ★

【题目描述】

> 有一个 $n(n \leq 10^6)$ 个结点的二叉树。给出每个结点的两个子结点编号（均不超过 n），建立一棵二叉树（根节点的编号为 1），如果是叶子结点，则输入 `0 0`。
>
> 建好这棵二叉树之后，请求出它的深度。二叉树的深度是指从根节点到叶子结点时，最多经过了几层。

【输入描述】

> 第一行一个整数 n，表示结点数。
>
> 之后 n 行，第 i 行两个整数 l 、 r ，分别表示结点 i 的左右子结点编号。若 $l = 0$ 则表示无左子结点， $r = 0$ 同理。

【输出描述】

> 一个整数，表示最大结点深度。

【示例一】

> 输入：
>
> 7
>
> 2 7
>
> 3 6
>
> 4 5
>
> 0 0
>
> 0 0
>
> 0 0
>
> 0 0
>
> 输出：
>
> 4

【解法】

二叉树的高度 = 1 + max(左子树的高度， 右子树的高度)；

因此，可以递归解决。

【参考代码】

```cpp
#include <iostream>

using namespace std;

const int N = 1e6 + 10;

int n;
int l[N], r[N];

int dfs(int root)
{
    if(!root) return 0;

    return max(dfs(l[root]), dfs(r[root])) + 1;
}

int main()
{
```

```
19      cin >> n;
20      for(int i = 1; i <= n; i++)
21      {
22          cin >> l[i] >> r[i];
23      }
24
25      cout << dfs(1) << endl;
26
27      return 0;
28  }
```

4.4 [NOIP2001 普及组] 求先序排列

> **题目来源：** 洛谷
>
> **题目链接：** P1030 [NOIP2001 普及组] 求先序排列
>
> **难度系数：** ★

【题目描述】

> 给出一棵二叉树的中序与后序排列。求出它的先序排列。（约定树结点用不同的大写字母表示，长度 ≤ 8）。

【输入描述】

> 共两行，均为大写字母组成的字符串，表示一棵二叉树的中序与后序排列。

【输出描述】

> 共一行一个字符串，表示一棵二叉树的先序。

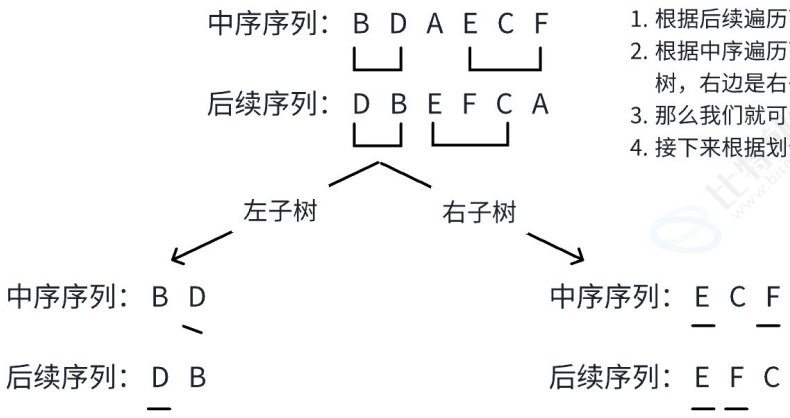
【示例一】

> 输入：
>
> BADC
>
> BDCA
>
> 输出：
>
> ABCD

【解法】

学习「二叉树」这种数据结构时会遇到的经典问题，一定要先学会在「演草纸」上「手动模拟」出结果，然后在模拟的基础编写我们的代码。

我们先用一个具体的例子模拟一下，在已知「中序」和「后序」的基础上，如何求「先序」：



1. 根据后续遍历可知，最后一个元素 A 就是当前这棵树的根
2. 根据中序遍历可知，根节点 A 会把整个序列分成两部分，左边是左子树，右边是右子树
3. 那么我们就可以在后序遍历的序列中，也找出对应的左右子树
4. 接下来根据划分情况，继续处理相应的子树

处理「左右子树」的方式与处理「原始序列」的方式一致，这样我们就可以用「递归」的方式求出先序序列。

【参考代码】

```cpp
#include <iostream>

using namespace std;

string a, b;

void dfs(int l1, int r1, int l2, int r2)
{
    // 递归出口
    // 当区间不合法时，返回
    if(l1 > r1) return;

    // 1. 确定根节点 - b[r2]
    cout << b[r2];

    int p = l1;
    while(a[p] != b[r2]) p++; // p 用来标记中序遍历中根节点的位置

    // 2. 划分左右子树
    dfs(l1, p - 1, l2, l2 + p - l1 - 1); // 递归处理左子树
    dfs(p + 1, r1, l2 + p - l1, r2 - 1); // 递归处理右子树
}

int main()
{
    cin >> a >> b;

    dfs(0, a.size() - 1, 0, b.size() - 1);
```

```
29
30
31        return 0;
32    }
```

4.5 American Heritage

> **题目来源：** 洛谷
>
> **题目链接：** P1827 [USACO3.4] 美国血统 American Heritage
>
> **难度系数：** ★

【题目描述】

> 农夫约翰非常认真地对待他的奶牛们的血统。然而他不是一个真正优秀的记账员。他把他的奶牛们的家谱作成二叉树，并且把二叉树以更线性的“树的中序遍历”和“树的前序遍历”的符号加以记录而不是用图形的方法。
>
> 你的任务是在被给予奶牛家谱的“树中序遍历”和“树前序遍历”的符号后，创建奶牛家谱的“树的后序遍历”的符号。每一头奶牛的姓名被译为一个唯一的字母。（你可能已经知道你可以在知道树的两种遍历以后可以经常地重建这棵树。）显然，这里的树不会有多于 26 个的顶点。

【输入描述】

> 第一行一个字符串，表示该树的中序遍历。
>
> 第二行一个字符串，表示该树的前序遍历。

【输出描述】

> 单独的一行表示该树的后序遍历。

【示例一】

> 输入：
>
> ABEDFCHG
>
> CBADEFGH
>
> 输出：
>
> AEFDBHGC

【解法】

解法同《求先序序列》，先手动模拟一下，然后找出「相同子问题」，「递归」求解。

【参考代码】

```cpp
#include <iostream>

using namespace std;

string a, b;

void dfs(int l1, int r1, int l2, int r2)
{
    // 递归出口
    if(l1 > r1) return;

    // 先处理左右子树
    // 1. 找根节点
    // 2. 划分左右子树
    int p = l1;
    while(a[p] != b[l2]) p++;

    dfs(l1, p - 1, l2 + 1, l2 + p - l1); // 处理左子树
    dfs(p + 1, r1, l2 + p - l1 + 1, r2); // 处理右子树

    // 输出根节点
    cout << b[l2];
}

int main()
{
    cin >> a >> b;

    dfs(0, a.size() - 1, 0, b.size() - 1);

    return 0;
}
```

4.6 二叉树问题

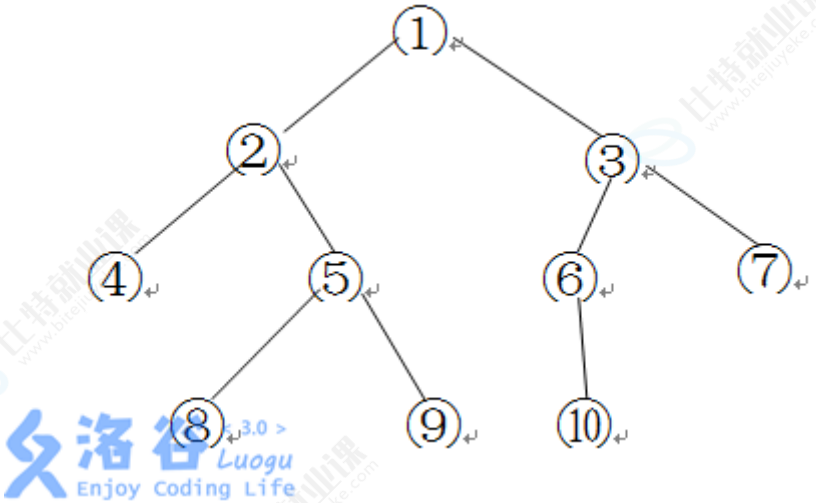
> **题目来源：** 洛谷
>
> **题目链接：** P3884 [JLOI2009] 二叉树问题
>
> **难度系数：** ★★

【题目描述】

> 如下图所示的一棵二叉树的深度、宽度及结点间距离分别为：
>
> - 深度：4

- 宽度： 4
- 结点 8 和 6 之间的距离： 8
- 结点 7 和 6 之间的距离： 3

其中宽度表示二叉树上同一层最多的结点个数，节点 u,v 之间的距离表示从 u 到 v 的最短有向路径上向根节点的边数的两倍加上向叶节点的边数。



给定一颗以 1 号结点为根的二叉树，请求出其深度、宽度和两个指定节点 x,y 之间的距离。

【输入描述】

第一行是一个整数，表示树的结点个数 n 。
接下来 $n-1$ 行，每行两个整数 u,v ，表示树上存在一条连接 u,v 的边。
最后一行有两个整数 x,y ，表示求 x,y 之间的距离。

【输出描述】

输出三行，每行一个整数，依次表示二叉树的深度、宽度和 x,y 之间的距离。

【示例一】

输入：

```
10
1 2
1 3
2 4
2 5
3 6
3 7
5 8
5 9
```

6 10

8 6

输出：

4

4

8

【解法】

深度：递归。

宽度：宽搜。

两点之间的距离：通过向上不断找父结点。第一个重叠的位置，就是两者的最近公共祖先。可以一边寻找，一边计算结果。

【参考代码】

```cpp
#include <iostream>
#include <vector>
#include <queue>

using namespace std;

const int N = 110;

int n;
vector<int> edges[N]; // 存树

int fa[N]; // fa[i] 表示 i 结点的父亲
int dist[N]; // dist[i] 表示 i 到 x 的最短距离

int dfs(int u)
{
    int ret = 0;
    for(auto v : edges[u])
    {
        ret = max(ret, dfs(v));
    }
    return ret + 1;
}

int bfs()
{
```

```cpp
    queue<int> q;
    q.push(1);

    int ret = 0;

    while(q.size())
    {
        int sz = q.size();
        ret = max(ret, sz);

        while(sz--)
        {
            int u = q.front(); q.pop();
            for(auto v : edges[u])
            {
                q.push(v);
            }
        }
    }

    return ret;
}

int main()
{
    cin >> n;
    for(int i = 1; i < n; i++)
    {
        int u, v; cin >> u >> v;
        // u -> v
        edges[u].push_back(v);
        fa[v] = u;
    }

    // 求深度
    cout << dfs(1) << endl;

    // 求宽度
    cout << bfs() << endl;

    // 求距离
    int x, y; cin >> x >> y;
    while(x != 1)
    {
        dist[fa[x]] = dist[x] + 1;
        x = fa[x];
    }
```

```
74
75      int len = 0;
76      while(y != 1 && dist[y] == 0)
77      {
78          len++;
79          y = fa[y];
80      }
81
82      cout << dist[y] * 2 + len << endl;
83
84      return 0;
85  }
```